

COLEGIO DE ESTUDIOS CIENTÍFICOS Y TECNOLÓGICOS DEL ESTADO DE MÉXICO

PLANTEL VILLA VICTORIA



CECYTEM

MANUAL TÉCNICO DEL SISTEMA CONECTA Y APRENDE



PROYECTO INTEGRADOR
ÉLITE STRATEGIC



FECHA DE ELABORACIÓN
11 DE JUNIO DE 2026



VERSIÓN
1.0



EQUIPO
ÉLITE STRATEGIC



ALIADO PRINCIPAL
DIF MUNICIPAL



EDUCAR, TRANSFORMAR, INCLUIR.
JUNTOS CONSTRUIMOS OPORTUNIDADES.

Datos del Proyecto

A continuación se presenta la información fundamental del proyecto en un formato estructurado para su rápida comprensión.

CATEGORIA	NOMBRE DEL PROYECTO	PROPOSITO	DESCRIPCION BREVE (TIEMPO, LUGAR, PARTICIPANTES)
			
Educación Digital y Desarrollo Humano 	Conecta y Aprende 	Desarrollar un sitio web educativo y comunitario que ofrezca contenidos formativos en habilidades digitales básicas, cultura, arte y oficios, gestionado por estudiantes en colaboración con DIF Municipal. 	El proyecto se desarrollará a 4 meses por estudiantes de quinto semestre de programación y está dirigido a instancias de apoyo a personas en rehabilitación por consumo de sustancias nocivas de centros de Alcohólicos Anónimos. Inicio 18 de marzo y primera versión 12 de junio de 2026. 

El enfoque principal del proyecto se centra en mitigar el desempleo y la falta de oportunidades laborales en la comunidad. Para ello, se crea una plataforma que capacite en habilidades digitales, oficios y desarrollo humano, fortaleciendo la inserción social de personas en rehabilitación.El proyecto se desarrolla en un ámbito académico y comunitario, con estudiantes de cuarto y quinto semestre como principales responsables del diseño y la implementación, y DIF Municipal como aliado para la difusión y apoyo social. La plataforma está pensada para ser útil tanto en entornos de rehabilitación como en comunidades vulnerables con brecha digital.

Resumen Ejecutivo

Conecta y Aprende es una plataforma web diseñada para gestionar procesos de educación digital y apoyo comunitario en contextos de rehabilitación, integrando módulos para solicitud de actividades, gestión de evidencias, administración financiera, emisión de certificados y comunicación interna, con el objetivo de apoyar a personas en proceso de recuperación y fortalecer el trabajo colaborativo entre estudiantes, docentes e instituciones sociales. El proyecto fue desarrollado por el equipo Élite Strategic como un trabajo integrador de cinco alumnos, en colaboración con DIF Municipal y centros de Alcohólicos Anónimos, y se construyó en un ciclo de cuatro meses hasta su primera versión presentada el 12 de junio de 2026, demostrando que una aplicación web puede transformar la gestión de programas de apoyo y mejorar la inclusión social de personas vulnerables. El sistema facilita un canal digital seguro para el registro y seguimiento de actividades formativas, permite a los participantes solicitar y obtener aprobaciones de actividades con claridad, registrar evidencias semanales y calcular el progreso individual, además de administrar transacciones económicas, pagos y préstamos con trazabilidad, y mantener una comunicación interna efectiva entre participantes, administradores y docentes. Esta plataforma aborda la problemática de desempleo y falta de oportunidades laborales agravada por la brecha digital y la ausencia de seguimiento estructurado, ofreciendo un entorno donde los usuarios acceden a contenidos educativos, registran tareas y evidencias, reciben retroalimentación administrativa y mantienen un historial transparente de su evolución, reduciendo el uso de procesos manuales y papel, evitando la pérdida de información y mejorando la trazabilidad de acciones clave. El manual técnico está pensado para desarrolladores, evaluadores, administradores y stakeholders, y recoge análisis de arquitectura, descripción de datos, diseño de interfaces, módulos implementados, requerimientos funcionales y no funcionales, seguridad, pruebas y recomendaciones, con ejemplos, tablas y narrativas detalladas que lo convierten en una referencia sólida.

1. Introducción

Conecta y Aprende es una plataforma web diseñada para apoyar programas de rehabilitación social y educación comunitaria. Su objetivo es brindar un espacio digital donde los participantes puedan solicitar actividades, registrar evidencias, acceder a recursos formativos y recibir seguimiento por parte de administradores y docentes.

El proyecto responde a la necesidad de combatir la exclusión laboral y la brecha de habilidades en personas que enfrentan procesos de rehabilitación por consumo de sustancias. Se busca además fortalecer el trabajo colaborativo entre estudiantes universitarios, instituciones sociales y beneficiarios del programa.

El manual técnico cubre la implementación de la aplicación, las herramientas utilizadas, la arquitectura, la modelación de datos y el diseño de interfaces. También documenta las responsabilidades del equipo, las pruebas realizadas y las recomendaciones para su continuidad.

Esta sección introductoria incluye el contexto del proyecto, la motivación detrás de su creación y los elementos que definen su impacto social y educativo. La plataforma se plantea como un puente entre la comunidad, el sector académico y los servicios de apoyo social.

A continuación se presentan los capítulos con la descripción del proyecto, los objetivos, la problemática abordada, las tecnologías empleadas y todos los detalles técnicos necesarios para comprender y mantener la solución. Cada apartado incluye ejemplos, tablas y explicaciones extensas para garantizar su utilidad como documento de referencia.

2. Descripción general del proyecto

El proyecto consiste en una aplicación web desarrollada con Flask, que centraliza la gestión de procesos de apoyo, educación y rehabilitación social. La plataforma agrupa funcionalidades de registro, autenticación, solicitud de actividades, manejo de evidencias, transacciones económicas y comunicación interna.

Los roles definidos en el sistema son: administrador, docente, alumno y persona en recuperación. Cada rol tiene acceso a funcionalidades diferenciales, lo que permite que el sistema sea flexible y seguro. Por ejemplo, el administrador controla la aprobación de solicitudes y el estado de las cuentas, mientras que la persona en recuperación puede solicitar actividades y subir evidencias.

El desarrollo del proyecto se realiza en un ciclo de cuatro meses, con estudiantes de quinto semestre de programación como responsables principales, en colaboración con la institución DIF Municipal y los centros de Alcohólicos Anónimos. La primera versión se presentó el 12 de junio de 2026, con un alcance funcional que cubre los procesos esenciales de seguimiento y administración del programa.

Uno de los valores agregados del proyecto es su enfoque en actividades formativas de oficios, cultura y habilidades digitales. La plataforma ofrece recursos y estructuras que permiten a los participantes aprender gradualmente, registrar su progreso y recibir retroalimentación. Esto convierte el sistema en una herramienta educativa y de inclusión social.

La aplicación utiliza una base de datos NoSQL para manejar información diversa sin un esquema rígido, lo que facilita la adaptación a los diversos tipos de contenido que se almacenan. Aunque gran parte de la lógica se encuentra en un solo módulo, la estructura de carpetas y la separación de recursos visuales permiten una implementación ordenada y mantenible.

A continuación se presenta un cuadro con los elementos clave del proyecto:

Elemento	Descripción
Nombre del proyecto	Conecta y Aprende
Duración	4 meses
Inicio	18 de marzo de 2026
Versión inicial	12 de junio de 2026
Versión final	Mes de Julio
Ubicación del desarrollo	Estudiantes de cuarto semestre de programación y DIF Municipal
Beneficiarios	Personas en rehabilitación por consumo de sustancias y la comunidad local

Este cuadro sintetiza el alcance temporal y organizativo del proyecto, destacando su carácter educativo y comunitario. El enfoque práctico permite a los estudiantes involucrarse de forma directa con un problema real de su entorno y aportar una solución tecnológica significativa.

3. Objetivos del sistema

El sistema tiene como objetivos principales los siguientes:

- Automatizar la gestión de usuarios y el control de accesos según roles definidos. Esto reduce errores administrativos y permite una administración diferenciada por perfil.
- Facilitar el registro, revisión y aprobación de solicitudes de actividades por parte de personas en rehabilitación. Se busca que el proceso sea claro, accesible y con respuestas del administrador.
- Permitir el seguimiento continuo mediante evidencia semanal, con notificaciones y cálculo de progreso. Esto ayuda a mantener el compromiso de los participantes y a monitorear su avance.

- Gestionar operaciones económicas como pagos, depósitos, retiros y préstamos de forma registrada y auditada. Es vital para controlar recursos y apoyar decisiones basadas en datos.
- Ofrecer una administración centralizada para el manejo de certificados, mensajes y reportes del sistema. La centralización mejora la visibilidad del programa y facilita la toma de decisiones.
- Brindar información clara y actualizada para cada perfil de usuario, con interfaces orientadas a sus necesidades. Esto mejora la experiencia y la usabilidad del sistema.

Además, el sistema busca establecer una base tecnológica que permita al equipo continuar con el desarrollo y la mejora del sistema en fases posteriores. A nivel académico, el objetivo es aprender buenas prácticas de desarrollo de software y generar una solución con impacto social real.

Los objetivos del sistema pueden agruparse en tres grandes bloques: gestión de usuarios y permisos, gestión de actividades y evidencias, y gestión administrativa de recursos y comunicación. Cada bloque corresponde a una necesidad concreta del proyecto y se implementa mediante funciones específicas en la aplicación.

4. Problemática que resuelve la plataforma

El principal problema identificado es el desempleo y la falta de oportunidades laborales en la comunidad, especialmente entre personas en rehabilitación. Esta situación se ve agravada por la brecha digital, la falta de formación en habilidades técnicas y la ausencia de un seguimiento formal de avances educativos.

La plataforma Conecta y Aprende aborda esta problemática ofreciendo un espacio digital donde los participantes pueden acceder a contenidos formativos en habilidades digitales básicas, cultura, arte y oficios. Además, se integra un

componente de seguimiento y evidencia que permite medir el progreso y aplicar sanciones suaves por falta de participación.

La solución también atiende a la necesidad de vincular a entidades sociales como DIF Municipal y centros de Alcohólicos Anónimos, para que estos actores puedan supervisar y apoyar a los beneficiarios. De este modo, se genera un entorno colaborativo entre estudiantes, instituciones y participantes.

Otro aspecto importante es la reducción de la carga administrativa: mediante el sistema se evita el manejo de datos en papel, correos informales y registros dispersos. Esto mejora la coherencia de la información y permite una gestión más eficiente de solicitudes y recursos.

En resumen, la plataforma resuelve:

- La falta de un canal digital estructurado para la capacitación y el seguimiento de personas en rehabilitación.
- La ausencia de un sistema unificado para gestionar solicitudes de actividad y los avances semanales.
- El déficit en registro y control de recursos económicos asignados a los participantes.
- La dificultad de comunicación entre beneficiarios, docentes y administradores en un mismo entorno digital.
- La necesidad de información clara sobre responsabilidades y roles del equipo que gestiona el proyecto.

5. Alcance del proyecto

El alcance del proyecto define los procesos cubiertos por la plataforma, así como las restricciones y lo que no se incluye en esta primera versión. El objetivo es establecer claramente los límites funcionales y técnicos del sistema.

El sistema cubre los siguientes procesos:

- Registro y autenticación de usuarios.

- Gestión y administración de roles y permisos.
- Solicitud y revisión de actividades formativas.
- Registro y evaluación de evidencias semanales.
- Gestión de transacciones de efectivo y pagos de actividades.
- Solicitud y administración de préstamos.
- Administración de certificados y notificaciones.
- Generación de reportes administrativos básicos.

No se incluye en esta versión:

- Integración con pasarelas de pago reales como Stripe o PayPal.
- Envío automático de correos electrónicos.
- Soporte multi-idioma avanzado.
- Inteligencia artificial o recomendación automática de actividades.

Esta delimitación permite centrar el desarrollo en las funcionalidades críticas para el programa, garantizando un producto viable en el tiempo de cuatro meses. Las mejoras adicionales pueden incorporarse en fases posteriores una vez estabilizado el sistema base.

A continuación se presenta una tabla con los entregables esperados en esta versión:

Entregable	Descripción
Sistema de autenticación	Registro, inicio de sesión y gestión de cuenta.
Panel administrativo	Gestión de usuarios, solicitudes, transferencias y reportes.
Dashboard de actividad	Seguimiento de evidencias, progreso y comunicación.
Módulo financiero	Registro de depósitos, retiros y pagos.
Módulo de certificados	Gestión y aprobación de certificados.

El alcance se ha definido de manera que cada entregable tenga valor funcional por sí mismo y contribuya a la experiencia integral del usuario. El producto resultante es un sistema usable y con potencial de ampliación.

6. Tecnologías utilizadas

La plataforma está construida con una selección de tecnologías que facilitan el desarrollo rápido y la escalabilidad. A continuación se describen cada una y su papel dentro del proyecto.

Python 3 es el lenguaje elegido para el desarrollo backend, gracias a su sintaxis clara y la disponibilidad de bibliotecas para web y bases de datos. Flask se utilizó como framework web ligero, lo que permite definir rutas y controladores de manera directa sin la complejidad de un framework de mayor tamaño.

MongoDB es la base de datos seleccionada por su carácter NoSQL y su capacidad para almacenar documentos flexibles. Esto es útil en un proyecto donde los datos pueden evolucionar y contener campos opcionales, como solicitudes de actividad con diferentes tipos de información adicional.

Jinja2 permite generar las páginas HTML dinámicamente a partir de plantillas, lo que simplifica la separación entre lógica y presentación. CSS se emplea para definir el estilo visual de la aplicación y mantener una identidad coherente en las distintas vistas.

Werkzeug se utiliza para el manejo seguro de contraseñas y la normalización de nombres de archivos subidos. La librería python-docx se emplea únicamente para generar este manual en formato Word; no forma parte de la aplicación web en producción.

A continuación se presenta una tabla con el conjunto tecnológico principal:

Componente	Tecnología	Uso principal
Backend	Python 3	Lógica del servidor y

		procesado de datos.
Framework web	Flask	Definición de rutas y controladores HTTP.
Base de datos	MongoDB	Almacenamiento de documentos y datos del sistema.
Plantillas	Jinja2	Generación dinámica de HTML.
Seguridad	Werkzeug	Hashing de contraseñas y manejo de archivos.
Frontend	HTML/CSS	Interfaz de usuario y estilos visuales.

La combinación de estas tecnologías permite un desarrollo ágil, con una curva de aprendizaje relativamente corta para estudiantes y con soporte para futuras adaptaciones. MongoDB aporta flexibilidad, mientras que Flask ofrece simplicidad para construir la lógica del sistema.

7. Arquitectura general del sistema

La arquitectura general de Conecta y Aprende se organiza en capas lógicas que separan la presentación, la lógica de negocio y los datos. Aunque la implementación se realiza en un único archivo principal, la estructura conceptual sigue un patrón claro de responsabilidades.

La capa de presentación está conformada por las plantillas Jinja2 situadas en ``templates/`` y los archivos de estilo en ``static/``. Estas plantillas reciben datos desde el backend y muestran la información al usuario. La capa de aplicación contiene las rutas Flask y la lógica de negocio implementada en ``app.py``. Finalmente, la capa de datos está representada por MongoDB y las operaciones realizadas con PyMongo.

En la práctica, la arquitectura se presenta de la siguiente manera:

Capa	Elemento	Descripción
Presentación	templates/	Vistas HTML y contenido dinámico.
Presentación	static/	CSS, imágenes, iconos y recursos estáticos.
Aplicación	app.py	Rutas, validaciones y lógica de negocio.
Configuración	config.py	Parámetros de entorno y conexión a MongoDB.
Persistencia	MongoDB	Almacenamiento de documentos y datos del sistema.

El servidor Flask inicializa la aplicación, carga la configuración y establece la conexión con MongoDB. Antes de la primera petición, se ejecuta una función de inicialización de la base de datos que crea índices y asegura la existencia de un usuario administrador predeterminado. Esto garantiza que el sistema esté listo para su uso desde el primer arranque.

Desde el punto de vista funcional, el sistema maneja diversas rutas: páginas públicas, rutas protegidas por autenticación, rutas administrativas y rutas específicas para el manejo de actividades, finanzas, mensajes y certificados. La separación por roles asegura que cada usuario vea solo las opciones pertinentes.

8. Descripción de la base de datos no relacional

MongoDB es la base de datos utilizada en el proyecto. Su modelo de documentos es especialmente apropiado para este sistema, ya que permite almacenar información heterogénea sin forzar un esquema rígido. Cada registro se representa como un documento BSON, similar a JSON, lo que facilita la lectura y la evolución de los datos.

El diseño aprovecha colecciones separadas para cada tipo de entidad: usuarios, solicitudes, evidencias, notificaciones, transacciones, préstamos, mensajes, configuraciones y publicaciones. Esto permite consultas eficientes y mantiene la organización lógica de la información.

MongoDB permite un crecimiento flexible del esquema. Por ejemplo, las solicitudes de actividad pueden incluir campos adicionales como ``program_notes``, ``meeting_info`` y ``weekly_plan``, sin que sea necesario modificar otras colecciones. Esto es útil en un proyecto en constante evolución donde los requisitos pueden ampliarse.

La base de datos se inicializa con índices clave en los campos utilizados con más frecuencia, como correos electrónicos y referencias de usuario. Estos índices mejoran el rendimiento de las consultas y aseguran integridad en campos únicos. Además, se emplea una colección de contadores para generar identificadores numéricos secuenciales, lo que facilita la generación de IDs legibles y ordenados.

El uso de MongoDB también permite un registro más natural de las evidencias y las transacciones, que pueden tener atributos opcionales como archivos adjuntos, comentarios y fechas de creación y actualización. Esta flexibilidad es ideal para modelar procesos sociales y educativos.

9. Estructura de colecciones o documentos

A continuación se describen las colecciones principales y sus campos más relevantes. La estructura de cada colección está diseñada para cubrir los requisitos del sistema y mantener coherencia entre los documentos.

9.1 Colección users

La colección `users` almacena los datos de todos los usuarios registrados. Cada documento contiene información de autenticación, perfil y estado de la cuenta. La clave `id` es única y se utiliza como referencia en otras colecciones.

Campos principales:

- id: número entero único.
- nombre: nombre completo del usuario.
- email: correo electrónico, único por usuario.
- password: hash seguro de la contraseña.
- role: rol del usuario (admin, docente, alumno, recuperacion).
- active: bandera de activación de la cuenta.
- created_at: fecha de registro.

Ejemplo de documento:

```
{
  "id": 1,
  "nombre": "Rubí Guadalupe Juan Juárez",
  "email": "admin@conectayaprende.local",
  "password": "<hash>",
  "role": "admin",
  "active": true,
  "created_at": "2026-06-11T..."
}
```

Esta colección es la piedra angular para el acceso seguro al sistema y la evaluación de permisos por rol. Mantener esta estructura clara es esencial para futuras ampliaciones.

9.2 Colección `activity_requests`

La colección `activity_requests` almacena las solicitudes de actividad creadas por los usuarios con rol de recuperación. Cada documento guarda el estado del trámite y las instrucciones asignadas por el administrador.

Campos principales:

- `id`: identificador de la solicitud.
- `user_id`: referencia al usuario solicitante.
- `nombre`: nombre del solicitante.
- `email`: correo del solicitante.
- `rol`: rol del usuario.
- `actividad`: nombre de la actividad solicitada.
- `descripcion`: descripción del motivo de la solicitud.
- `disponibilidad`: horario o condiciones de disponibilidad.
- `adicional`: información adicional opcional.
- `estado`: pendiente, aceptado o rechazado.
- `admin_instructions`: instrucciones administrativas.
- `program_notes`: notas de programación de la actividad.
- `meeting_info`: información de reuniones o citas.
- `start_date`: fecha de inicio del plan.
- `weekly_plan`: plan semanal de actividades.
- `penalty_dates`: semanas con penalizaciones.
- `created_at`: fecha de creación.
- `updated_at`: fecha de actualización.

Esta colección soporta la gestión completa del ciclo de vida de una solicitud, desde su creación hasta su resolución. Permite al administrador documentar instrucciones y programaciones específicas para cada caso.

9.3 Colección notifications

La colección `notifications` registra los avisos internos enviados a los usuarios. Esto incluye notificaciones de aprobación, rechazo, penalizaciones o actualizaciones de certificados. La marca `leido` permite controlar si el usuario ha consultado el mensaje.

9.4 Colección activity_evidences

La colección `activity\_evidences` almacena las evidencias semanales que los usuarios presentan. Cada registro contiene texto descriptivo y opcionalmente una imagen subida. Estas evidencias son la base para calcular el progreso y las semanas de cumplimiento.

9.5 Colección cash_transactions

La colección `cash\_transactions` registra transacciones relacionadas con depósitos, retiros, pagos y descuentos. Cada documento tiene un estado que permite al administrador controlar su avance, desde pendiente hasta completado o rechazado.

Campos importantes: `tipo`, `cantidad`, `estado`, `referencia`, `comentario`, `actividad`, `ticket\_number`, `created\_at` y `updated\_at`

9.6 Colección action_history

Esta colección almacena un historial de acciones importantes realizadas por los usuarios. Sirve para auditar el sistema y revisar quién realizó cada acción, cuál fue el tipo de evento y cuándo sucedió. Su uso es crítico para transparencia y seguimiento interno.

9.7 Colección loans

La colección `loans` contiene las solicitudes de préstamos. Aquí se registran los montos solicitados, los intereses, el plazo de pago y el estado de la solicitud. Esta información es útil para la administración financiera y el seguimiento del apoyo otorgado.

9.8 Colección `contact_messages`

La colección `contact_messages` guarda los mensajes enviados por los usuarios a la administración. Incluye datos de contacto, asunto, contenido del mensaje y la respuesta del administrador. Este registro permite mantener un historial completo de la comunicación interna.

9.9 Colección `system_settings`

La colección `system_settings` almacena configuraciones del sistema en formato clave-valor. Actualmente se utiliza para registrar la fecha activa del sistema, lo que permite definir una fecha de referencia distinta a la fecha real del servidor para pruebas o seguimientos específicos.

9.10 Colección `posts`

Esta colección facilita la publicación de artículos o entradas de blog dentro de la aplicación. Aunque es un módulo secundario, contribuye a la difusión de contenidos y noticias relevantes para los usuarios.

9.11 Colección `certificates`

La colección `certificates` registra los certificados emitidos o pendientes. Incluye información sobre el usuario, la solicitud de actividad asociada, el estado del certificado y la fecha en que fue otorgado o actualizado.

10. Diseño de interfaces desarrolladas

Las interfaces del sistema se diseñaron para ser claras, funcionales y accesibles. Se utiliza una plantilla base con un encabezado fijo y un menú de navegación que se adapta según el rol del usuario.

Las páginas se dividen en varias categorías: acceso y autenticación, gestión de perfil, panel de actividad, administración y recursos. Cada categoría está construida para facilitar tareas específicas y mejorar la experiencia del usuario.

A continuación se muestra una tabla con las principales vistas y su propósito:

Vista	Archivo	Propósito
Página de inicio	home.html	Mostrar información principal y accesos rápidos según el rol.
Login	login.html	Permite al usuario iniciar sesión en el sistema.
Registro	register.html	Formulario para crear una cuenta nueva.
Perfil	profile.html	Visualiza los datos del usuario y su estado.
Actividad	activity_dashboard.html	Panel de seguimiento de evidencias, finanzas y progreso.
Solicitudes administrativas	admin_activity_requests.html	Permite al administrador revisar y procesar solicitudes.

El diseño emplea tarjetas, tablas y formularios para ordenar la información de forma intuitiva. La interfaz del dashboard de actividad está estructurada en secciones que muestran la actividad actual, el progreso, las evidencias cargadas y las opciones financieras.

La navegación se adapta al rol: el administrador tiene acceso a controles adicionales para gestionar usuarios y contenidos, mientras que los usuarios en recuperación se centran en su seguimiento y solicitudes de actividades. Este diseño por perfiles facilita el uso y evita confusiones.

Se han implementado mensajes de estado y validaciones en pantalla para informar al usuario sobre el resultado de sus acciones, como la creación de solicitudes, el envío de evidencias o el cambio de estado de una transacción. Estos elementos son esenciales para una experiencia de usuario adecuada.

11. Módulos implementados hasta el momento

El desarrollo actual del proyecto incluye múltiples módulos funcionales. A continuación se presenta un panorama detallado de cada uno, junto con ejemplos de funciones y comportamientos implementados.

Módulo de autenticación y gestión de usuario:

- Login y logout con verificación de credenciales.
- Registro de nuevos usuarios con credenciales hasheadas.
- Edición de perfil, cambio de nombre y contraseña.

Módulo administrativo:

- Listado de usuarios y gestión de activaciones.
- Creación, edición y eliminación de cuentas desde el panel administrativo.
- Revisión de mensajes de contacto y respuesta administrativa.

Módulo de solicitudes de actividad:

- Envío de solicitudes con datos de actividad, disponibilidad y descripción.
- Procesamiento por parte del administrador con aceptación o rechazo.
- Asignación de planes semanales y notas de seguimiento.

Módulo de evidencias y progreso:

- Subida de evidencias semanales con texto y archivo opcional.
- Cálculo de progreso mediante semanas registradas.
- Generación de penalizaciones por faltas de evidencia.

Módulo financiero:

- Solicitud de depósitos y retiros con registro de transacciones.
- Pago de actividades con registro de referencias y comentarios.
- Estado administrativo de cada transacción.

Módulo de certificados:

- Creación de solicitudes de certificados al completar progreso.
- Aprobación o rechazo de certificados por el administrador.
- Notificación al usuario del estado del certificado.

Módulo de comunicación:

- Mensajería interna a través de notificaciones y mensajes de contacto.
- Registro de mensajes y respuestas administrativas.

Este modularizado funcional ofrece una visión clara de lo que se ha implementado y permite priorizar mejoras futuras. Cada módulo corresponde a un área de responsabilidad del sistema y puede escalar de forma independiente.

12. Requerimientos funcionales

Los requerimientos funcionales definen lo que el sistema debe hacer. Para este proyecto se han identificado los siguientes elementos:

ID	Requerimiento	Descripción
RF1	Registro de usuarios	Permitir crear cuentas con nombre, correo, contraseña y rol.
RF2	Autenticación	Permitir iniciar y cerrar

		sesión de forma segura.
RF3	Gestión de usuarios	Permitir al administrador ver, editar y activar/desactivar cuentas.
RF4	Solicitud de actividades	Permitir a usuarios en rehabilitación enviar solicitudes de actividad.
RF5	Revisión de solicitudes	Permitir al administrador aceptar o rechazar solicitudes.
RF6	Registro de evidencias	Permitir subir evidencia semanal con texto e imagen.
RF7	Cálculo de progreso	Calcular avance basado en evidencias entregadas.
RF8	Gestión financiera	Registrar pagos, depósitos y retiros con estado.
RF9	Solicitudes de préstamo	Permitir solicitar préstamos y gestionar su estado.
RF10	Notificaciones	Enviar avisos internos sobre cambios y respuestas.
RF11	Certificados	Administrar certificados de progreso con aprobación.

Los requerimientos funcionales describen las capacidades imprescindibles del sistema. Cada uno de ellos se traduce en una o varias rutas y vistas dentro de la aplicación.

13. Requerimientos no funcionales

Los requerimientos no funcionales establecen criterios de calidad y comportamiento del sistema. Son esenciales para asegurar la experiencia de usuario y la estabilidad del software.

ID	Requerimiento	Descripción
RNF1	Accesibilidad	Ser accesible desde navegadores modernos.
RNF2	Performance	Responder de forma rápida a las interacciones de los usuarios.
RNF3	Seguridad	Proteger contraseñas y datos sensibles.
RNF4	Integridad de datos	Mantener consistencia mediante validaciones e índices.
RNF5	Mantenibilidad	Escribir código claro y documentado.
RNF6	Escalabilidad	Permitir la adición de nuevos módulos con facilidad.
RNF7	Robustez	Manejar entradas inválidas y errores de forma controlada.
RNF8	Usabilidad	Ofrecer una experiencia

		clara y orientada por rol.
--	--	----------------------------

Los requisitos de calidad complementan la funcionalidad principal. Aseguran que la plataforma no solo funcione, sino que también sea segura, intuitiva y sostenible a mediano plazo.

14. Seguridad y control de acceso

La seguridad del sistema se centra en dos aspectos: la protección de credenciales y la separación de permisos por rol. El uso de hash para contraseñas evita el almacenamiento de datos sensibles en texto plano, reduciendo el riesgo en caso de una vulneración.

El control de acceso se realiza mediante la sesión Flask, que guarda información como ``user_id`` y ``role``. Las rutas sensibles verifican la presencia de un usuario autenticado y validan que el rol tenga permiso para la operación solicitada.

Por ejemplo, solo los administradores pueden acceder a las rutas de gestión de usuarios, de solicitud de certificados y de actualización de transacciones financieras. Los usuarios con rol de recuperación pueden solicitar actividades y subir evidencias, pero no pueden ver ni modificar datos de otros usuarios.

Para el almacenamiento de archivos se utiliza ``secure_filename()`` y se valida la extensión permitida. Esto protege contra nombres maliciosos y archivos con extensiones no deseadas. También se maneja el error de conexión a la base de datos con una vista de error HTTP 503, lo que mejora la robustez del sistema.

Adicionalmente, la gestión de roles se documenta en los arreglos ``ROLE_PAGES`` y ``ROLE_ACTIONS``, que definen qué páginas y acciones están disponibles para cada perfil. Esto hace que el control de acceso sea explícito y fácil de revisar.

15. Proceso de instalación y despliegue

El proceso de instalación describe los pasos necesarios para poner en marcha el sistema en un entorno local o de pruebas. Se detallan los requisitos previos y los comandos esenciales.

Requisitos previos:

- Python 3 instalado en el equipo.
- MongoDB instalado y ejecutándose localmente o de forma remota.
- Acceso al repositorio del proyecto y permisos para instalar dependencias.

Pasos de instalación:

1. Clonar o copiar el repositorio del proyecto.
2. Crear un entorno virtual: ``python -m venv venv``.
3. Activar el entorno virtual: ``venv\Scripts\activate`` en Windows.
4. Instalar dependencias: ``pip install -r requirements.txt`` o ``pip install flask pymongo python-docx``.
5. Configurar ``config.py`` con la URI de MongoDB y la base de datos.
6. Ejecutar la aplicación: ``python app.py``.

Si se desea inicializar la base de datos, se puede ejecutar ``create_mysql_db.py``, que en realidad prepara índices y configuraciones iniciales en MongoDB.

Después, se puede acceder a la aplicación en el navegador utilizando la dirección local indicada por Flask.

Para producción se recomienda usar un servidor WSGI y un proxy reverso, así como almacenar ``SECRET_KEY`` y las credenciales de MongoDB en variables de entorno en lugar de en el código fuente.

16. Metodología de desarrollo

La metodología aplicada combina trabajo en equipo, división de roles y ciclos iterativos de desarrollo. El proyecto avanzó a través de fases definidas, con entregables parciales y revisiones periódicas.

Fases principales:

- Análisis: identificación de requisitos y problemáticas del entorno.
- Diseño: definición de arquitectura, estructura de datos e interfaces.
- Desarrollo: implementación de la lógica, vistas y persistencia de datos.
- Pruebas: validación funcional y verificación de los flujos del sistema.
- Documentación: generación del manual técnico y guías de uso.

El equipo trabajó de forma colaborativa, con roles asignados para liderar el proyecto, programar el sistema, investigar el campo y probar la solución. Esto permitió avanzar con claridad y asegurar que cada parte del proyecto estuviera alineada con el objetivo general.

Se recomienda que el trabajo futuro mantenga esta dinámica: definir tareas concretas, establecer plazos, documentar decisiones y revisar avances en reuniones periódicas.

17. Estrategia de pruebas y validación

La estrategia de pruebas se enfoca en verificar la funcionalidad, la usabilidad y la robustez del sistema. Se contemplan pruebas manuales y casos de uso reales que cubran los aspectos más críticos del proyecto.

- Pruebas funcionales: verificar cada formulario, ruta y acción del sistema.
- Pruebas de roles: comprobar que cada perfil accede solo a sus funciones.
- Pruebas de validación: ingresar datos inválidos y verificar la respuesta adecuada.
- Pruebas de flujo: seguir el proceso completo desde registro hasta la aprobación de solicitudes.
- Pruebas de transacciones: registrar depósitos, retiros y pagos con distintos estados.

Además, se recomienda documentar los casos de prueba y los resultados, de modo que cualquier cambio futuro pueda verificarse con mayor rapidez. Esta

estrategia mejora la confianza en el software y reduce la probabilidad de errores en producción.

18. Mantenimiento y futuras mejoras

El mantenimiento del sistema implica mantener el código organizado, corregir errores y actualizar dependencias. A continuación se indican recomendaciones para el mantenimiento y la evolución del proyecto:

- Modularizar el código separando rutas, servicios y utilidades.
- Agregar pruebas automatizadas para los módulos críticos.
- Documentar cambios y mantener un registro de versiones.
- Revisar y actualizar dependencias de Python y MongoDB.
- Mejorar gradualmente la interfaz de usuario y la accesibilidad.

Futuras mejoras posibles: integración con servicios de correo, pagos en línea, dashboards analíticos, soporte multicanal y ampliación de los módulos de formación. También es recomendable incorporar un módulo de gestión de contenidos para publicar recursos dinámicos y materiales de capacitación.

19. Roles y responsabilidades del equipo

Nombre del equipo: **Élite Strategic**

19.1 Alan Jesús Escobar Almazán

Rol: Líder del Proyecto.

Responsabilidades:

- Coordinar al equipo y asegurar el cumplimiento de plazos y entregables.
- Asignar tareas en función de las fortalezas y el progreso del equipo.
- Supervisar la calidad del desarrollo y la documentación.
- Mantener comunicación con los docentes y las instancias de apoyo involucradas.
- Gestionar la planificación y la distribución del trabajo.

19.2 Rubí Guadalupe Juan Juárez

Rol: Programador y Documentador.

Responsabilidades:

- Implementar el código del sistema en Python y Flask.
- Construir la conexión con MongoDB y definir las operaciones de datos.
- Elaborar documentación técnica clara y completa.
- Registrar decisiones de diseño y mantener actualizada la documentación.
- Apoyar al tester en la revisión y corrección de errores.

19.3 María Guadalupe Contreras Reyes

Rol: Investigador de Campo.

Responsabilidades:

- Recopilar información del contexto comunitario y las necesidades de los usuarios.
- Analizar la problemática de desempleo y brecha digital.
- Proponer ideas y requerimientos basados en el entorno real.
- Documentar hallazgos que orienten el desarrollo del proyecto.
- Verificar que el sistema responda a las expectativas de los beneficiarios.

19.4 Ytzel Melissa Martínez

Rol: Desarrollador.

Responsabilidades:

- Diseñar y construir las funcionalidades del software.
- Integrar distintos módulos y asegurar su compatibilidad.
- Optimizar la calidad y organización del código.
- Colaborar con el programador y el tester para resolver problemas técnicos.
- Asegurar que el sistema entregue una experiencia coherente para el usuario.

19.5 Ulises Contreras García

Rol: Tester.

Responsabilidades:

- Probar el software para identificar errores y fallos.
- Verificar el funcionamiento de cada módulo y flujo.
- Registrar incidencias y colaborar en su resolución.
- Asegurar que la experiencia de usuario sea adecuada y funcional.
- Coordinar pruebas de regresión cuando se realizan cambios.

20. Glosario de términos

Este glosario define términos clave utilizados en el proyecto para asegurar una comprensión uniforme entre los miembros del equipo y los evaluadores.

- Flask: microframework Python para desarrollar aplicaciones web.
- MongoDB: base de datos NoSQL orientada a documentos.
- Jinja2: motor de plantillas para generar HTML dinámico.
- Colección: conjunto de documentos en MongoDB similar a una tabla en bases relacionales.
- Documento: registro almacenado en MongoDB, equivalente a un objeto JSON.
- Rol: perfil de usuario que define permisos y acceso a funcionalidades.
- Evidencia: registro que prueba el avance de una actividad semanal.
- Transacción: movimiento financiero registrado en el sistema.
- Certificado: reconocimiento oficial del cumplimiento de un módulo o actividad.

21. Conclusión y recomendaciones finales

El equipo de Élite Strategic presenta esta primera versión del manual técnico como un documento colectivo que recoge el esfuerzo realizado en el desarrollo de Conecta y Aprende, su propósito social y los componentes principales de la plataforma. Esta edición inicial describe la arquitectura, los procesos clave, las tecnologías utilizadas y las responsabilidades del equipo, y se entiende como un punto de partida para el seguimiento, la mejora continua y la evolución del proyecto.

Durante el proceso de trabajo se enfrentaron dificultades reales y debates técnicos, desde la definición de los roles y las rutas hasta la organización de los datos en MongoDB, pasando por la adaptación de la solución a los requerimientos sociales del proyecto. A pesar de esas discusiones y de que no todo fue perfecto, el equipo supo avanzar, aprender de los errores y superar los bloqueos para construir una primera versión funcional y coherente.

El documento refleja tanto los logros como las lecciones aprendidas, y deja clara la intención de continuar mejorando. El equipo propone mantener la documentación actualizada, iterar sobre el diseño, incorporar nuevas funcionalidades y fortalecer la calidad técnica en cada etapa, con el fin de consolidar Conecta y Aprende como una herramienta cada vez más sólida para la educación digital, la rehabilitación social y la inserción comunitaria.